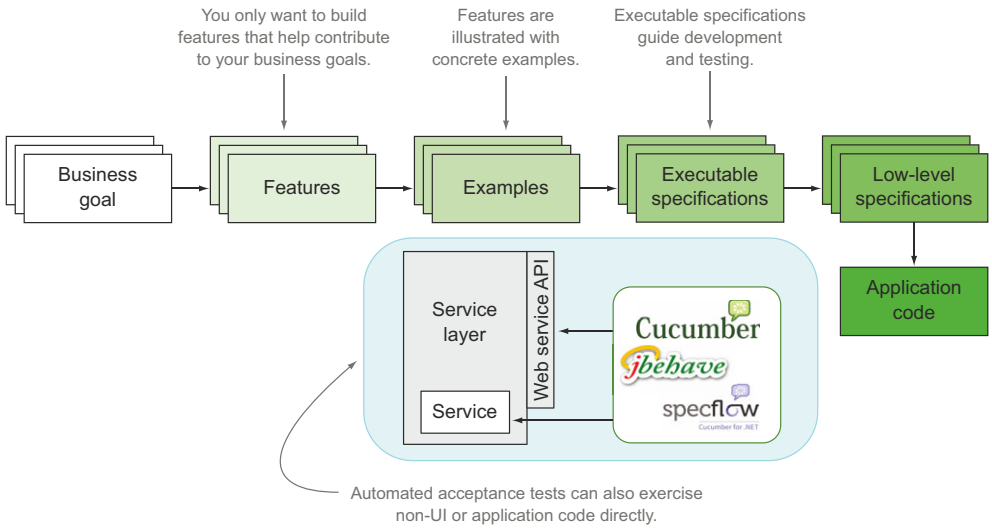


Figure 9.1 In this chapter we'll focus on automating acceptance tests that exercise the non-UI components of your application.

In chapter 8, you learned how to automate acceptance tests using automated web testing tools such as Selenium 2/WebDriver. Web tests are a great way to simulate the user's journey through the system, to illustrate interactions with an application, and to document the key features of your application. Stakeholders can easily relate to web tests, as they're highly visual, intuitive, and map closely to the user experience. Automated web tests can also be used to demonstrate new features, which is a great way to increase confidence in the tests. Automated web tests are also the only way to effectively test business logic that's implemented directly within the user interface (UI).

But you also saw that end-to-end web tests tend to execute significantly more slowly than tests that don't involve the UI. Interacting with a browser adds significant overhead in terms of execution time and system resources. Web tests that rely on a real browser are also more subject to technical or environment-related issues that are hard to control. For example, web tests can fail because the wrong version of a browser is installed on a test machine, or because the browser crashes. Tests that fail for reasons unrelated to the application logic waste development time and resources, and can reduce the team's confidence in the test suite.

Although they can have great value, web tests shouldn't be your only option when it comes to automating your BDD scenarios:

- Most applications need a judicious mix of both automated UI tests and automated tests for non-UI components.
- Non-UI tests can work at different levels of the application, including tests for the controller layer of an MVC application, tests that exercise the business rules implemented in the application code directly, and tests that work with remote services.